Verificação de hash nas produções Apple e Google — conhecimento do A-Ward e do G-Ward

21/10/2026 · @Gise RJ

Para que serve

Apple e Google mandam, junto com a produção, uma lista com o hash de cada arquivo. Conferir a entrega contra essa lista é a única prova de que o conteúdo é o que o provedor produziu, porque é o único documento que nenhuma das partes redigiu.

Este documento reúne o que o A-Ward (Apple) e o G-Ward (Google) aprenderam lendo essas listas em casos reais. Cada regra abaixo veio de um defeito medido, não de suposição.

	Apple (A-Ward)	Google (G-Ward)
Onde está o hash	`account-download-details.csv` (manifesto, dentro do pacote inicial)	PDF "Valores de Hash dos Arquivos Produzidos" (um ou vários)
Formato	CSV com colunas nomeadas	texto corrido extraído de PDF
Algoritmo	SHA-256 (sempre) e SHA-512 (quando a coluna vem)	SHA-512
O que é coberto	cada `.zip.gpg` cifrado e, às vezes, o `.zip` decifrado	só os arquivos produzidos; volumes, Carta e Anexos ficam de fora
Leitura	`csv.DictReader`	`pypdf` + expressão regular

Os trechos de código citados são do A-Ward `core.py` e do G-Ward `src/processar.py`. O validador da Defesa (G-Vouch, `valores_hash.py`) copia o parser do G-Ward.

Apple: o manifesto account-download-details.csv

Na Apple o hash não precisa ser raspado de documento: ele vem em colunas de um CSV. O trabalho é achar o CSV, ler as colunas certas e conferir em dois pontos, antes e depois da decifração.

Onde o CSV está

1. A Apple manda um pacote inicial `<caso>_production_<AAAAMMDDhhmmss>.zip.gpg` (link e senha chegam em e-mails separados).
2. Decifrado e extraído, ele traz o manifesto `<caso>_<AAAAMMDDhhmmss>-account-download-details.csv`, o PDF do pedido e a planilha APL.
3. O A-Ward procura recursivamente qualquer arquivo que termine em `account-download-details.csv` (`preparar_pacote_inicial`). Não dependa do nome exato: o prefixo muda por caso.
4. O número do caso pode ser tirado do nome do CSV pelo padrão `^(\d{6,})_(\d{14})-account-download-details\.csv$`. Já houve pacote sem PDF e sem APL (AC06_08), e o CSV era a única fonte do número.

Colunas

Coluna	Conteúdo	Observação
`Data_Type`	categoria (Backup, Photos, Mail…)	
`File_Name`	`<conta_id>-...-BACKUP-....zip.gpg`	o `conta_id` é o número antes do primeiro `-`; `-BACKUP-` vai para Backup, o resto para Blended
`File_Link`	URL de download	expira
`GPG_Size`	texto como `1.5 GB`	converter para bytes; aceita vírgula decimal
`GPG_SHA256`	hash do `.zip.gpg` cifrado	sempre presente; coluna obrigatória
`GPG_SHA512`	idem, em SHA-512	coluna nova; pode faltar
`CLEAR_TEXT_SHA256` / `CLEAR_TEXT_SHA512`	hash do `.zip` já decifrado	só em parte das produções; pode vir vazio
`Link Expiry Date`	`MM-DD-AAAA`	formato americano

Regras de leitura que o A-Ward aplica (`ler_csv_detalhes`):

- abrir com `encoding="utf-8-sig"`: o arquivo pode ter BOM, e sem isso a primeira coluna vira `Data_Type`;

- exigir só o mínimo (`Data_Type` , `File_Name` , `File_Link` , `GPG_Size` , `GPG_SHA256`) e tolerar colunas novas — a Apple já acrescentou colunas sem aviso;
- normalizar todo hash com `.strip().lower()` antes de comparar.

As duas conferências

1. **GPG:** baixado o `.zip.gpg` , calcula-se o hash do arquivo cifrado e compara-se com `GPG_SHA256` (ou `GPG_SHA512` , se o usuário escolheu SHA-512). Só o que confere segue para a decifração.
2. **CLEAR_TEXT:** depois de decifrar, se a coluna CLEAR_TEXT tiver valor, calcula-se o hash do `.zip` e compara-se. Sem valor, o estado registra `sem_hash` e não `OK` .

Divergiu na primeira vez, o A-Ward baixa de novo, uma vez. Divergiu de novo, ele registra que o arquivo servido pela Apple difere do manifesto: isso já não é tropeço de rede, é prova.

Arquivo que já existe no destino só é reaproveitado se o hash conferir; senão é apagado e baixado de novo.

O que o A-Ward grava

- `HASH\hash_validation_GPG_<caso>.xlsx|.csv` e `hash_validation_ZIP_<caso>.xlsx|.csv` , com colunas Nome, Algoritmo, Hash fornecido, Hash calculado, `is_valid` , inválidos no topo;
- `HASH\hash_sha256_GPG_<caso>.txt` no formato `sha256sum` (`<hash> *<conta>/<arquivo>` , fim de linha LF), para conferir com `sha256sum -c` , HashMyFiles ou 7-Zip;
- por conta, `hash_CONSOLIDADO_<conta>.xlsx` , com as duas pontas (GPG e CLEAR_TEXT) na mesma linha;
- `CONTAS_IDENTIFICADAS\verificacao_pasta_HASH_<caso>.txt` , o selo da pasta HASH, gravado fora dela.

Armadilhas da Apple

- **Linha sem hash no manifesto.** O teste do pipeline é `if esperado and calc != esperado` , então linha sem hash passa para a decifração; na planilha ela sai `is_valid = False` . No seu app, trate a ausência de hash como estado próprio, nunca como `OK` .
- **CRLF na lista conferível.** O `sha256sum` lê o `\r` como parte do nome e falha em todas as linhas. Gravar com `newline="\n"` .
- **Caminho da lista.** O `.zip` decifrado fica em `<conta>/Backup/` ou `<conta>/Blended/` ; sem a subpasta, a conferência externa não acha o arquivo.

- **Apagar os `.gpg` depois de decifrar** não pode levar junto a prova. Por isso os relatórios ficam em `HASH\`, fora das pastas de dados.
- **Multi-CPU.** Cada CPU vê só as próprias contas; o relatório geral do caso só é gravado na consolidação, por uma única instância.

Google: o PDF "Valores de Hash dos Arquivos Produzidos"

Na Google o hash está num PDF de texto corrido, e precisa ser raspado. A âncora confiável é a sequência `<nome do arquivo>:SHA512- <128 hex>`; o resto do texto é ruído a remover na ordem certa.

Onde o PDF está

- Fica entre os arquivos baixados do LERS, com "Valores de Hash" no nome.
- **Pode haver vários.** A Google parte a relação quando a produção é grande. Na solicitação 150483767 vieram quatro (`[1]` a `[4]`, com 750, 348, 757 e 760 nomes). Leia todos, ordenados por nome, e una os mapas.
- O PDF relaciona **só os arquivos produzidos** (os zips por conta e produto, e também `.csv`, `.pdf`, `.png` do Waze). Os volumes `<n>-<AAAAMMDD>-<k>.zip`, a Carta e os Anexos não têm hash declarado: no caso 148876058, 534 nomes e nenhum de volume.

O texto que sai do pypdf

Cabeçalho, título e o primeiro nome saem colados, sem separador (exemplo fictício):

```
Autos n. 5001234-56.2026.4.02.5101Ref. Interna: 150483767Valores de Hash dos
Arquivos Produzidos123fulano@gmail.com.987654321.GoogleChat_001.zip:SHA512-
3f9a...(128 hex)
```

E ao fim de cada página repete-se o rodapé da Google ("Google LLC", "1600 Amphitheatre Parkw", "Mountain View, California 94043", "www.google.com").

O parser, passo a passo

1. Extrair o texto de todas as páginas e concatenar.
2. **Remover primeiro o cabeçalho** com `Autos n\..*?Ref\. Interna:\s*\d+` (flag `re.S`).
3. Só depois remover o título e o rodapé (substituição de texto literal).
4. Capturar pares com `([A-Za-z0-9._@%+\-]+\.[A-Za-z0-9]{2,5}):SHA512-\s*([0-9a-fA-F]{128})`.

5. Limpar o início de cada nome: tirar hex de 16+ caracteres colado (resto do hash anterior numa quebra de página) e restos como `Produzidos` ou `Ref.Interna:<n>`.
6. Guardar `{nome: hash.lower()}`.

```
txt = re.sub(r"Autos n\..*?Ref\. Interna:\s*\d+", "", txt, flags=re.S)
for lixo in ("Google LLC", "1600 Amphitheatre Parkw",
             "Mountain View, California 94043", "www.google.com",
             "Valores de Hash dos Arquivos Produzidos"):
    txt = txt.replace(lixo, "")
for nome, h in re.findall(
        r"([A-Za-z0-9._@%+\-]+\.[A-Za-z0-9]{2,5}):SHA512-\s*([0-9a-fA-F]
{128})", txt):
    nome = re.sub(r"^[0-9a-f]{16,}", "", nome)
    nome = re.sub(r"^(Produzidos|Ref\.?Interna:?\d*)", "", nome)
    mapa[nome] = h.lower()
```

Por que a ordem importa: se o título sair antes, o `\d+` da Ref. Interna encosta no primeiro nome e engole os dígitos iniciais dele. Bug real: `123gremista...@gmail.com` virou `gremista...@gmail.com`, e o arquivo saiu ao mesmo tempo como FALTANDO e EXTRA.

O mesmo PDF declara os autos e a solicitação. Leia-os do texto **cru**, antes de qualquer limpeza, com `Autos n\.\s*(.+?)\s*Ref\. Interna:\s*(\d+)`, e confronte com o caso em que o PDF foi achado.

Nome do arquivo e conta

- O nome casa com o PDF **exatamente como a Google o deu**. Nunca renomeie o arquivo; organize por pasta.
- A conta sai do nome: tudo antes do primeiro token numérico (`<email>.<id da conta>.<Produto>...`); sem id (Waze, `<email>.Waze.pdf`), até o TLD.
- **Nomes longos vêm truncados com `...`**, no arquivo e no PDF (`carlosnogueirat...tescn@gmail.com`). A fonte exata do nome completo é a linha `Originating Identifier` do `ExportSummary` dentro do zip. Sem ela, casar prefixo e sufixo contra as contas solicitadas, sem caixa e sem repetir, e **só resolver com candidato único**; havendo dúvida, declarar a ambiguidade.
- **Não fundir contas por ponto.** O Gmail ignora pontos, mas a Google honra o identificador literal pedido. Fundir suprimiu uma cobrança legítima.

O que o G-Ward grava

- `HASH\Hash_geral_<caso>.xlsx|.csv` e `Hash_<conta>.xlsx` para cada conta, inclusive a que não recebeu nada; colunas Arquivo, Conta, Algoritmo, Hash fornecido (PDF), Hash calculado, Confere, Observação;

- `RELATORIO\completude_por_conta_<caso>.xlsx|.csv`, com Esperados (PDF), Entregues, Íntegros, Falha de hash, Faltando, Extras e Status;
- `HASH\Hash_IPJ.txt`, SHA-256 de tudo que está em `HASH\`, exceto ele mesmo;
- a minuta de reenvio à Google, quando há arquivo listado e não entregue.

Regras comuns de comparação

A comparação é a parte simples; o que dá errado é classificar. Confronte nos dois sentidos (listado e não entregue; entregue e não listado) e dê a cada arquivo exatamente um veredito.

Veredito	Condição	Gravidade
ÍNTEGRO (OK)	presente, hash igual ao declarado	—
ALTERADO (FALHA)	presente, hash diferente	grave
AUSENTE (FALTANDO)	declarado pelo provedor, não localizado	ressalva; pode gerar cobrança
EXTRA	presente, não declarado	grave
NÃO VERIFICÁVEL	ilegível, volume que não abriu, ou linha sem hash declarado	ressalva

Os nomes entre parênteses são os do G-Ward; os outros, do G-Vouch.

Regras

- **Normalizar antes de comparar:** `strip().lower()` no hash declarado e hexdigest minúsculo no calculado. O nome do arquivo é comparado exato, sem caixa alterada.
- **Calcular em blocos** (1 a 4 MB por `read`), nunca o arquivo inteiro em memória: há zips de dezenas de GB. Para arquivo dentro de zip, dá para passar o fluxo de `ZipFile.open()` direto ao hash, sem extrair.
- **AUSENTE só com os volumes íntegros.** Se algum volume do caso não abre, o que foi listado e não apareceu vira NÃO VERIFICÁVEL: não há como separar dano na cópia recebida de arquivo nunca produzido, e essa diferença decide se cabe cobrar o provedor.
- **Arquivo entregue com hash errado continua entregue.** Contá-lo como "sem entrega" seria afirmar ao provedor algo falso.

- **Lista ilegível não passa em silêncio.** Se um dos PDFs (ou o CSV) não pôde ser lido, tudo que só ele declarava sairia como EXTRA. Devolva a lista de falhas e marque a conferência como incompleta.
- **Conta listada e sem nenhum arquivo** precisa aparecer no relatório (com zero entregues), senão some exatamente o que mais interessa.
- **Não afirme falta do que foi pedido** só com os arquivos da produção: a lista de contas solicitadas está fora deles (na página do LERS, no ofício).

Duas famílias de hash, que não se misturam

- **Hash de custódia** (SHA-256 no G-Ward), calculado no momento do recebimento. Prova o que chegou.
- **Hash de conferência** (o algoritmo da lista do provedor), confrontado com o declarado. Prova que é o que o provedor produziu.

Fechar a cadeia

- Grave o resultado em CSV/XLSX e numa lista em formato `sha256sum` (`<hash> *<caminho>`, LF), que ferramentas de terceiros conferem sozinhas.
- Grave à parte o hash das próprias planilhas (o "selo": `verificacao_pasta_HASH` no A-Ward, `Hash_IPJ.txt` no G-Ward), sem incluir o próprio arquivo de selo.
- Grave o SHA-256 da própria lista do provedor (PDF ou CSV): ela não se autentica sozinha, e isso permite confrontar a cópia de uma parte com a da outra.
- O selo protege contra corrupção e troca acidental, não contra adulteração deliberada. Seus hashes devem ir para o documento assinado (Informação de Extração).
- Dê a etapa por concluída só quando o selo existir. Um caso já foi marcado como feito com o disco cheio e o selo nunca escrito.

Erros que já cometemos e casos de teste

Os defeitos abaixo aconteceram em produção real e cada um deveria virar um teste no seu app antes da primeira entrega.

Defeito	Onde	Efeito medido	Correção
Só o primeiro PDF de hash era lido	G-Ward, até 08/10/2026	sol. 150483767: 750 OK e 1.865 EXTRA (71% da produção acusada de não relacionada); a união dos 4 PDFs bateu 2.615 de 2.615	ler todos e unir

Título removido antes do cabeçalho	G-Ward	`123gremista...` virou `gremista...`; um arquivo saiu como FALTANDO e EXTRA	cabeçalho primeiro
Corte no primeiro `" - "` para tirar prefixo	G-Ward, até v0.6.6	`conta@x.com.123.Chat - Backup_001.zip` perderia a conta e iria para SEM_CONTA	casar o prefixo inteiro (`^\d{6,12} - `)
Fusão de contas por ponto	G-Ward v0.6.6.1 (descartada)	suprimiu cobrança legítima	não fundir
Conta truncada comparada sem tratamento	G-Ward	conta real acusada de não entregue e a truncada dada como desconhecida	ExportSummary; prefixo+sufixo com candidato único
Número do caso só procurado no PDF	A-Ward, até v0.4.2.1	pacote sem PDF saiu `SEM_NUMERO` nos arquivos de hash	ler também o nome do CSV
Lista conferível com CRLF	A-Ward	`sha256sum -c` falha em todas as linhas	gravar LF

Checklist para o novo app

- [ ] Acha o CSV da Apple por sufixo `account-download-details.csv`, recursivamente
- [ ] Lê o CSV com `utf-8-sig` e tolera colunas novas
- [ ] Confere o `.zip.gpg` contra `GPG_SHA256` / `GPG_SHA512` e o `.zip` contra `CLEAR_TEXT_*` quando houver
- [ ] Trata linha sem hash como NÃO VERIFICÁVEL, nunca como ÍNTEGRO
- [ ] Lê todos os PDFs "Valores de Hash" do caso e une os mapas
- [ ] Remove o cabeçalho antes do título
- [ ] Aceita qualquer extensão no nome (não só `.zip`)
- [ ] Não renomeia arquivos
- [ ] Resolve conta truncada pelo ExportSummary e declara ambiguidade
- [ ] Confronta nos dois sentidos e lista conta sem entrega
- [ ] Rebaixa AUSENTE para NÃO VERIFICÁVEL quando há volume ilegível
- [ ] Avisa quando um PDF/CSV não pôde ser lido
- [ ] Calcula hash em blocos

- [ ] Grava lista `sha256sum` (LF), planilha e selo fora da pasta selada

Casos de teste mínimos

1. CSV com BOM e com uma coluna desconhecida a mais: deve ler normalmente.
2. CSV com uma linha de `GPG_SHA256` vazio: o arquivo não pode sair ÍNTEGRO.
3. Texto de PDF com cabeçalho colado num nome que começa com dígitos (`...Ref. Interna: 150483767Valores de Hash dos Arquivos Produzidos123conta@gmail.com...`): o nome deve sair com o `123` .
4. Dois PDFs com metade dos nomes cada: nenhum EXTRA.
5. Um dos dois PDFs corrompido: alerta de conferência incompleta.
6. Nome truncado `abc...xyz@gmail.com` com uma candidata e com duas: resolve na primeira, declara ambiguidade na segunda.
7. Arquivo com um byte trocado: ALTERADO, e a conta continua contada como entregue.
8. Um volume truncado no caso: o faltante sai NÃO VERIFICÁVEL; sem o volume, sai AUSENTE.
9. Lista `sha256sum` gerada pelo app conferida com `sha256sum -c` ou 7-Zip: todas OK.

O G-Vouch tem as provas dos vereditos em `prova_vereditos.py` , e o G-Ward as dos PDFs múltiplos em `AUDITORIA\provas\prova_15_varios_pdfs_de_hash.py` ; vale pedir cópia como ponto de partida.